

AI ASSISTED CODING

LAB ASSIGNMENT – 11.4

HALLTICKETNO:2403A52395

BATCH: 14

TASK 1:

class Stack:

```
    """A simple Stack implementation using a Python list."""
```

```
    def __init__(self):
```

```
        """Initializes an empty Stack."""
```

```
        self._items = []
```

```
    def push(self, item):
```

```
        """
```

```
        Adds an item to the top of the stack.
```

```
        Args:
```

```
            item: The item to be added to the stack.
```

```
        """
```

```
        self._items.append(item)
```

```
    def pop(self):
```

```
        """
```

```
        Removes and returns the item from the top of the stack.
```

```
        Returns:
```

```
            The item from the top of the stack.
```

Raises:

IndexError: If the stack is empty.

"""

if not self.is_empty():

 return self._items.pop()

else:

 raise IndexError("pop from empty stack")

def peek(self):

"""

Returns the item at the top of the stack without removing it.

Returns:

The item at the top of the stack.

Raises:

IndexError: If the stack is empty.

"""

if not self.is_empty():

 return self._items[-1]

else:

 raise IndexError("peek from empty stack")

def is_empty(self):

"""

Checks if the stack is empty.

Returns:

True if the stack is empty, False otherwise.

```
"""
```

```
return len(self._items) == 0
```

```
# Test the Stack implementation
```

```
stack = Stack()
```

```
print(f"Is stack empty? {stack.is_empty()}")
```

```
stack.push(10)
```

```
stack.push(20)
```

```
stack.push(30)
```

```
print(f"Is stack empty? {stack.is_empty()}")
```

```
print(f"Top element: {stack.peek()}")
```

```
print(f"Pop element: {stack.pop()}")
```

```
print(f"Top element after pop: {stack.peek()}")
```

```
print(f"Pop element: {stack.pop()}")
```

```
print(f"Pop element: {stack.pop()}")
```

```
print(f"Is stack empty? {stack.is_empty()}")
```

```
try:
```

```
    stack.pop()
```

```
except IndexError as e:
```

```
    print(f"Caught expected error: {e}")
```

```
try:
```

```
stack.peek()

except IndexError as e:

    print(f"Caught expected error: {e}")
```

OUTPUT:

```
Is stack empty? True
Is stack empty? False
Top element: 30
Pop element: 30
Top element after pop: 20
Pop element: 20
Pop element: 10
Is stack empty? True
Caught expected error: pop from empty stack
Caught expected error: peek from empty stack
```

EXPLANATION:

implements a basic Stack data structure in Python using a list. Here's a breakdown:

- **class Stack::** This defines a new class named Stack.
- **__init__(self)::** This is the constructor of the class. It initializes an empty list `_items` which will store the elements of the stack. The underscore prefix `_` is a convention to indicate that `_items` is intended for internal use within the class.
- **push(self, item)::** This method adds an item to the top of the stack. It uses the `append()` method of the list, which adds the item to the end of the list. In a stack, the end of the list is considered the "top".
- **pop(self)::** This method removes and returns the item from the top of the stack. It first checks if the stack is empty using `is_empty()`. If not empty, it uses the `pop()` method of the list without an index, which removes and returns the last element (the top of the stack). If the stack is empty, it raises an `IndexError`.
- **peek(self)::** This method returns the item at the top of the stack without removing it. It also checks if the stack is empty. If not empty, it accesses the last element of the list using `self._items[-1]`. If the stack is empty, it raises an `IndexError`.
- **is_empty(self)::** This method checks if the stack is empty by checking the length of the `_items` list. It returns `True` if the length is 0, and `False` otherwise.

The code also includes test cases to demonstrate how to use the Stack class and its methods, including handling the IndexError when trying to pop or peek from an empty stack.

TASK 2:

class Queue:

```
    """A simple Queue implementation using a Python list."""
```

```
    def __init__(self):
```

```
        """Initializes an empty Queue."""
```

```
        self._items = []
```

```
    def enqueue(self, item):
```

```
        """
```

```
        Adds an item to the rear of the queue.
```

```
        Args:
```

```
            item: The item to be added to the queue.
```

```
        """
```

```
        self._items.append(item)
```

```
    def dequeue(self):
```

```
        """
```

```
        Removes and returns the item from the front of the queue.
```

```
        Returns:
```

```
            The item from the front of the queue.
```

```
        Raises:
```

```
            IndexError: If the queue is empty.
```

```

"""

if not self.is_empty():
    # Removing from the front of a list can be inefficient
    return self._items.pop(0)
else:
    raise IndexError("dequeue from empty queue")


def is_empty(self):
    """
    Checks if the queue is empty.

    Returns:
        True if the queue is empty, False otherwise.
    """
    return len(self._items) == 0


# Test the Queue implementation
queue = Queue()

print(f"Is queue empty? {queue.is_empty()}")

queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)

print(f"Is queue empty? {queue.is_empty()}")
print(f"Dequeue element: {queue.dequeue()}")
print(f"Dequeue element: {queue.dequeue()}")

```

```
print(f"Dequeue element: {queue.dequeue()}")
```

```
print(f"Is queue empty? {queue.is_empty()}")
```

```
try:
```

```
    queue.dequeue()
```

```
except IndexError as e:
```

```
    print(f"Caught expected error: {e}")
```

OUTPUT:

```
Is queue empty? True
Is queue empty? False
Dequeue element: 10
Dequeue element: 20
Dequeue element: 30
Is queue empty? True
Caught expected error: dequeue from empty queue
```

EXPLANATION:

implements a basic Queue data structure in Python using a list. Here's a breakdown:

- **class Queue::** This defines a new class named Queue.
- **__init__(self)::** This is the constructor of the class. It initializes an empty list `_items` which will store the elements of the queue.
- **enqueue(self, item)::** This method adds an item to the rear of the queue. It uses the `append()` method of the list, adding the item to the end. In this list-based implementation, the end of the list represents the rear of the queue.
- **dequeue(self)::** This method removes and returns the item from the front of the queue. It first checks if the queue is empty using `is_empty()`. If not empty, it uses `self._items.pop(0)` to remove and return the element at index 0, which is the front of the list and thus the front of the queue. Removing from the beginning of a list can be slow for large lists because all subsequent elements need to be shifted. If the queue is empty, it raises an `IndexError`.
- **is_empty(self)::** This method checks if the queue is empty by checking the length of the `_items` list. It returns `True` if the length is 0, and `False` otherwise.

The code also includes test cases to show how to use the Queue class and its methods, including handling the IndexError when trying to dequeue from an empty queue.

TASK 3 :

Test Case 1: Insert at the end and traverse

```
print("Test Case 1:")  
  
my_list = LinkedList()  
  
my_list.insert_at_end(10)  
my_list.insert_at_end(20)  
my_list.insert_at_end(30)  
  
my_list.traverse()
```

Test Case 2: Delete a value

```
print("\nTest Case 2:")  
  
my_list = LinkedList()  
  
my_list.insert_at_end(10)  
my_list.insert_at_end(20)  
my_list.insert_at_end(30)  
my_list.insert_at_end(20)  
my_list.insert_at_end(40)  
  
print("Original list:")  
  
my_list.traverse()  
  
my_list.delete_value(20)  
  
print("After deleting first 20:")  
  
my_list.traverse()  
  
my_list.delete_value(20)  
  
print("After deleting second 20:")  
  
my_list.traverse()
```



```
my_list.delete_value(50)
print("After deleting 50 (not in list):")
my_list.traverse()
my_list.delete_value(10)
print("After deleting 10 (head):")
my_list.traverse()
my_list.delete_value(40)
print("After deleting 40 (last):")
my_list.traverse()
my_list.delete_value(30)
print("After deleting 30 (only node):")
my_list.traverse()
```

Test Case 3: Delete from an empty list

```
print("\nTest Case 3:")
my_list = LinkedList()
print("Empty list:")
my_list.traverse()
my_list.delete_value(10)
print("After attempting to delete from empty list:")
my_list.traverse()
```

Output :



Test Case 1:

10 -> 20 -> 30 -> None

Test Case 2:

Original list:

10 -> 20 -> 30 -> 20 -> 40 -> None

After deleting first 20:

10 -> 30 -> 20 -> 40 -> None

After deleting second 20:

10 -> 30 -> 40 -> None

After deleting 50 (not in list):

10 -> 30 -> 40 -> None

After deleting 10 (head):

30 -> 40 -> None

After deleting 40 (last):

30 -> None

After deleting 30 (only node):

None

Test Case 3:

Empty list:

None

After attempting to delete from empty list:

None

Explanation :

1. Node Class:

- o This class represents a single element (or node) within the linked list.
- o `_init_(self, data)`: The constructor takes data as input, which is the value the node will hold. It initializes the node's data attribute and sets the next attribute to None. The next attribute is a pointer that will link this node to the next node in the sequence.

2. LinkedList Class:

- o This class represents the entire linked list.
- o `_init_(self)`: The constructor initializes the head attribute to None. The head is a pointer to the first node in the list. An empty list has no head, hence None.
- o `insert_at_end(self, data)`: This method adds a new node with the given data to the end of the list.

? It first creates a new_node.

- ? If the list is empty (self.head is None), the new_node becomes the head.
- ? If the list is not empty, it traverses the list starting from the head until it finds the last node (the one whose next is None).
- ? It then updates the next pointer of the last node to point to the new_node.
- o delete_value(self, value): This method deletes the first node it finds with the given value.
- ? It handles the case where the list is empty.
- ? It checks if the head node contains the value and updates the head if it does.
- ? If the value is not in the head, it traverses the list with current_node until it finds the node before the one to be deleted.
- ? If the node to be deleted is found, it updates the next pointer of the current_node to skip the node with the value, effectively removing it from the list.
- o traverse(self): This method iterates through the linked list from the head to the end, printing the data of each node. It prints "None" at the end to indicate the end of the list.

The code then provides three test cases (Test Case 1, Test Case 2, and Test Case 3) that demonstrate how to use these methods and verify their correctness.

0 / 2000

TASK 4:

Test with a list of integers

bst= BST()

elements_to_insert = [50, 30, 20, 40, 70, 60, 80]

for element in elements_to_insert:

bst.insert(element)

print("In-order traversal:")

print(bst.inorder_traversal())

Test search for present and absent elements

print("\nTesting search:")

present_element = 40

absent_element = 90

```
search_present = bst.search(present_element) search_absent = bst.  
search(absent_element)  
if search_present:  
    print(f"Search for {present_element}: Found")  
else: print(f"Search for {present_element}: Not Found")  
if search_absent:  
    print(f"Search for {absent_element}: Found")  
else: print(f"Search for {absent_element}: Not Found")  
Output :
```



In-order traversal:
[20, 30, 40, 50, 60, 70, 80]

Testing search:
Search for 40: Found
Search for 90: Not Found

EXPLANATION:

1. **Node Class:** This class represents a single node within the BST. Each node stores a key (the value of the node) and references to its left and right children.
2. **BST Class:** This class implements the Binary Search Tree itself.
 - `__init__`: Initializes an empty tree with the root set to None.
 - `insert(self, key)`: This public method is used to add a new node with the given key to the tree. It calls a recursive helper method `_insert_recursive` to find the correct position for the new node.
 - `_insert_recursive(self, current_node, key)`: This is a private helper method that recursively traverses the tree to insert the new node based on the BST property (left child's key is less than the parent's, and the right child's key is greater).

- `search(self, key)`: This public method searches for a node with the given key in the tree. It calls a recursive helper method `_search_recursive`.
- `_search_recursive(self, current_node, key)`: This is a private helper method that recursively traverses the tree to find the node with the specified key. It returns the node if found, otherwise returns `None`.
- `inorder_traversal(self)`: This public method performs an inorder traversal of the BST, which visits the left subtree, then the current node, then the right subtree. This traversal method for a BST visits the nodes in ascending order of their keys. It calls a recursive helper method `_inorder_recursive`.
- `_inorder_recursive(self, current_node, result)`: This is a private helper method that recursively performs the inorder traversal and appends the keys to a result list.

3. **Testing Code:** This part of the notebook demonstrates how to use the BST class.

- An instance of BST is created.
- A list of integers (`elements_to_insert`) is defined and then inserted into the BST using a loop and the `insert()` method.
- The `inorder_traversal()` method is called and the resulting sorted list of keys is printed.
- The `search()` method is tested with both a present element (40) and an absent element (90) to show that it correctly identifies whether a node exists in the tree. The output confirms that 40 is found and 90 is not.

In summary, the code provides a functional implementation of a Binary Search Tree, allowing you to insert elements, search for them, and traverse the tree in sorted order.

TASK 5:

Create a graph instance

```
g = Graph()
```

Add edges to the graph

```
g.add_edge(0, 1)
```

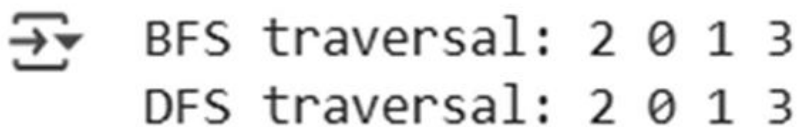
```
g.add_edge(0, 2)
```

```
g.add_edge(1, 2)
```

```
g.add_edge(2, 0)
```

```
g.add_edge(2, 3)
g.add_edge(3, 3)
# Perform BFS traversal
print("BFS traversal:", end=" ")
g.bfs(2)
print() # Print a newline after BFS output
# Perform DFS traversal
print("DFS traversal:", end=" ")
g.dfs(2)
print() # Print a newline after DFS output
```

Output:

A terminal window with a blue border showing the output of the code. It contains two lines of text: "BFS traversal: 2 0 1 3" and "DFS traversal: 2 0 1 3". To the left of the text is a small icon of a square with a right-pointing arrow inside.

```
➡ BFS traversal: 2 0 1 3
DFS traversal: 2 0 1 3
```

Explanation : This code demonstrates how to use the Graph class, which you've implemented earlier, to create a graph, add edges, and perform both Breadth-First Search (BFS) and Depth-First Search (DFS) traversals.

Here's a breakdown:

1. `g = Graph()`: This line creates an instance of your Graph class, effectively initializing an empty graph.
2. `g.add_edge(0, 1)`, `g.add_edge(0, 2)`, etc.: These lines call the `add_edge` method of the `g` object to add undirected edges between the specified nodes. For example, `g.add_edge(0, 1)` adds an edge connecting node 0 and node 1.
3. `print("BFS traversal:", end=" ")`: This line simply prints the label "BFS traversal: " to the console. The `end=" "` prevents a newline character from being printed, so the BFS output will appear on the same line.
4. `g.bfs(2)`: This line calls the `bfs` method on the graph `g`, starting the Breadth-First Search traversal from node 2. The output of the BFS traversal (the nodes visited in order) will be printed by the `bfs` method itself.

5. `print()`: This prints a newline character to separate the BFS output from the DFS output.

6. `print("DFS traversal:", end=" ")`: Similar to the BFS print statement, this prints the label "DFS traversal: " without a newline at the end.

7. `g.dfs(2)`: This line calls the `dfs` method on the graph `g`, starting the Depth-First Search traversal from node 2. The output of the DFS traversal will be printed by the `dfs` method.

8. `print()`: This prints a final newline character after the DFS output. In essence, this code block sets up a small example graph and then shows how to traverse it using the two implemented algorithms (BFS and DFS) starting from a specific node (node 2).